

# 개발자랑 일하는 PM을 위한 클로드 코드 — Larry's class 정리

"How를 알려주는 곳은 많은데, Why를 알려주는 곳은 별로 없다.  
래리는 후자를 알려주신다."

- **일시:** 2026-05-16 (토) 10:00-18:00
- **장소:** 인천 신포동 「사진의 도서관」
- **강사:** 김상희(Larry) — 아스켈 CEO, 20년+ 개발 경력, 클로드 코드 1년+ 실무 사용 (병의원 세무 백오피스 개발 중)
- **참가자:** 9명 (PM·마케터·개발자·사업가 혼합)
- **참가비:** 10만원 (대관료 + 점심 피자 포함)
- **컨셉:** "앱"을 만드는 클로드 코드가 아니라, **개발자랑 같이 일하는 PM이 알아야 할 그라운드**를 다루는 수업

## 1. 이 수업이 다른 클코 강의와 달랐던 점

비개발자용 클로드 코드 강의를 시중에 많지만, 대부분은 **"이렇게 치면 이게 나와요"**에서 멈춥니다.

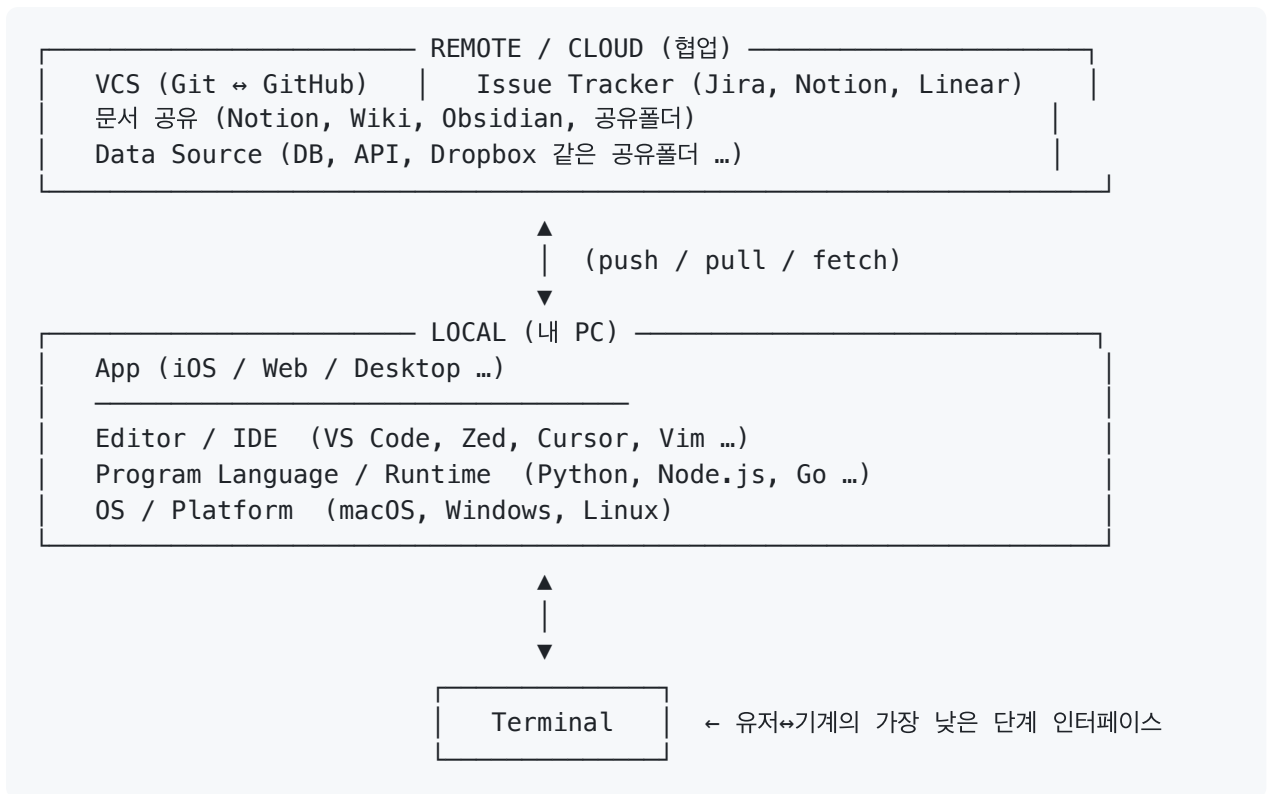
래리의 수업은 그 한 층 아래로 내려가서, 개발자들이 **암묵지로 깔고 일하는 용어·구조·습관**을 PM이 직접 먼저 보게 합니다.

- 클코를 켜기 전에, 터미널·홈브루·git·OS·런타임이 **서로 어떤 층에 있는지** 먼저 그려본다.
- 명령어 하나를 칠 때마다 **"내가 지금 어디에 있고( `pwd` ), 뭐가 있고( `ls -al` ), 누구한테 뭘 시키고 있는지( `claude` )"를 입으로 확인한다.**
- 옆 사람과 결과가 다르면, **"이게 정답이다"가 아니라 어디서 갈라졌는지를** 함께 추적한다.

그래서 이 수업의 진짜 산출물은 코드가 아니라, **개발자랑 같은 어휘로 말할 수 있는 PM**입니다.

## 2. 큰 그림 — 어디서부터 어디까지가 PM의 영역인가

래리가 화이트보드에 그려준 지도:



## 핵심 통찰

- **클로드 데스크탑 앱 vs 클로드 코드:**  
앱은 코딩부터 위쪽만 만진다. 클로드 코드는 터미널을 통해 OS까지 모두 만진다.  
→ 그래서 클로드 코드는 강력한 만큼 위험하고 (DB도 지울 수 있다), 그래서 **터미널 어휘를 배워야 한다**.
- **로컬은 내가 통제할 수 있는 영역, 리모트는 협업을 위한 영역.**  
PM이 가장 자주 헛갈리는 지점이다.

### 3. 세션별 진행

#### 세션 1 (10:00 ~ 12:00) — 환경 셋업과 개념 정렬

| 토픽                            | 핵심                                                                                                                                                                                                                                                                      |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Editor / IDE / Runtime / OS   | 4층 구조로 머릿속에 고정. 유행은 다 따라가지 않아도, 한 층씩 classic을 박아두면 10년 가도 안 흔들린다.                                                                                                                                                                                                       |
| 터미널                           | Ghostty 설치 (대안: iTerm, WezTerm, Waveterm). 터미널 종류는 카메라 브랜드처럼 취향.                                                                                                                                                                                                        |
| Homebrew                      | 맥 패키지 매니저 = "개발자들의 앱스토어". 설치도 클코한테 시키면 된다.                                                                                                                                                                                                                              |
| Claude Code 실행                | <code>pwd</code> → <code>ls -al</code> → <code>claude</code> 의 3박자. "내가 어디에 있는지 손으로 외우는" 머슬 메모리.                                                                                                                                                                        |
| 권한 (rwx)                      | 클코는 기본적으로 read만 자유, write/execute는 매번 물어봄. YOLO 모드는 정말 자신 있을 때만.                                                                                                                                                                                                        |
| <code>.claude/</code> 폴더 구조   | settings.json, skills, plugins, sessions, todos, telemetry, plans, agents — 옆 사람과 한 줄씩 소리내서 비교.                                                                                                                                                                         |
| 모델 선택 ( <code>/model</code> ) | Opus 4.7 / 1M Context / Sonnet 등. 1M은 책 한 권 분량이지만, 길어지면 10번 중 2-3번은 needle을 못 찾는다. 기본은 자동 스위칭이 알아서. 매번 모델을 의식적으로 선택하라.                                                                                                                                                  |
| <code>/effort</code>          | low / medium / high / xhigh. 권고는 최소 xhigh — 성능 곡선이 거의 평탄해지는 sweet spot.                                                                                                                                                                                                 |
| Plugins                       | <code>claude-plugins-official</code> 마켓플레이스에서 시작. 설치 예시: <code>playwright</code> , <code>pyright-lsp</code> , <code>superpowers</code> , <code>explanatory-output-style</code> , <code>code-simplifier</code> , <code>code-review</code> , <code>commit-commands</code> |

#### 래리가 반복해서 강조한 물

- "내가 어디 있는지 모르면 어떤 결과도 못 믿는다." → `pwd` 강박적으로.
- "실행 창/ 실행 내용에서 궁금한 것을 물어보는 창 : Multi 창을 잘 활용하라." → 화면 split 을 적절하게 사용할 것.
- "옆 개발자 셋업을 그대로 따라가라." → 내 취향보다 협업자의 셋업이 우선. 같이 일하는 사람이 좋아하는 색, 좋아하는 터미널, 좋아하는 단축키.

#### 세션 2 (12:30 ~ 14:00) — TDD로 만든 첫 함수, 그리고 인텐트

##### 실습 1 — `add(a, b)` 함수를 TDD로

처음에는 "두 수를 더하는 프로그램을 만들어줘"라고 던졌더니, 사람마다 결과가 다 달랐다.

- 누군가는 `add("가", "나") = "가나"` 가 그냥 통과
- 누군가는 유니코드 변수명까지 받아들임
- 누군가는 `0 + 5 = 8` 같은 엉뚱한 답

여기서 TDD가 등장한 이유:

"PM으로서 좋은 테스트 케이스를 만들 수만 있으면, 클로드가 안에서 어떤 짓을 하든 그 테스트만 통과시키면 동작하는 프로그램이다.

테스트만 보고도 스펙이 보인다 — 오히려 스펙 문서보다 명확하다."

그래서 같이 만든 룰:

- 두 수 = 숫자만. 문자열이 들어오면 `TypeError`
- **바운더리 테스트** 활용 — 1~10만까지 다 테스트할 게 아니라, *경계 (음수/0/한 자리/두 자리/오버플로/NaN/Inf)* 만 잡으면 나머지는 대수적으로 OK
- 매 커밋 전 `pre-commit` 훅으로 테스트 돌리기

실제로 만든 파일들:

```
excel-reader/
├─ add.py           # add() + 타입 검증
├─ test_add.py      # pytest parametrize로 8가지 케이스
├─ test_boundary.py # IEEE 754 inf/NaN, 64bit 경계, 부동소수점 정밀도
└─ boundary_demo.py # 경계값을 시각적으로 보여주는 데모 스크립트
```

### Intent — 프롬프트 엔지니어링의 본질

| 용어     | 의미                      |
|--------|-------------------------|
| Query  | 검색어 (SQL 등)             |
| Prompt | LLM 지시                  |
| Intent | 위 두 가지 모두에 깔린 <i>의도</i> |

프롬프트가 들어오면 LLM은 먼저 **인텐트 분석기**를 돌린다.

"내가 원하는 결과 ≠ 클로가 분석한 인텐트" 일 때 결과가 어긋난다.

**인텐트를 명확히 적어주면 stochastic이 줄어든다.** = 프롬프트 엔지니어링의 본질.

## 세션 3 (15:00 ~ 17:00) — 엑셀 리더, deterministic vs non-deterministic

### 실습 2 — 급여대장.xlsx 를 읽는 파이썬 프로그램 (TDD 기반)

급여대장 엑셀에는 두 가지 시트가 있었다:

- **급여대장-1단** : 평범한 1행 헤더
- **급여대장-2단** : 멀티컬럼 헤더 (과세/비과세 그룹핑, A4 출력용 디자인)

PM이 클코에게 시킨 흐름:

1. 라이브러리 리서치 — "내 손으로 검색하지 말고, 별 많이 받은 거 3개만 비교해줘" → `openpyxl` 채택

2. 급여대장-1단 읽는 함수 작성 → TDD로 검증
3. 다른 포맷( 급여대장-2단 )도 같은 함수가 처리할 수 있게 확장 → TDD가 빛나는 순간
4. 통계 ( payroll\_stats.py ) — 실수령액, 야근비율, 조직 단위 집계
5. CSV 출력 ( payroll\_export.py ) — UTF-8 BOM으로 엑셀에서 한글 깨지지 않게
6. CLAUDE.md 자동 생성으로 다음 세션이 이 프로젝트 컨텍스트를 바로 이해할 수 있게

왜 LLM이 다 읽지 않고 굳이 Python 프로그램을 만드는가?

|            | LLM에 통째로 던지기                | Python 툴로 빼기                   |
|------------|-----------------------------|--------------------------------|
| 100개 파일 처리 | 토큰 폭발 + 매번 결과 흔들림           | 한번 짜놓으면 거의 공짜                  |
| 정확도        | stochastic (운 나쁘면 숫자 1개 틀림) | deterministic (같은 인풋 → 같은 아웃풋) |
| 적합한 작업     | 트렌드 해석, 요약, 그레이 영역 판단       | 월말결산, 급여계산, 숫자가 절대 틀리면 안 되는 것  |

"숫자 잘 읽는 건 기계가 이미 잘한다. PM이 LLM한테 시켜야 하는 건 *얼추 맞으면 되는, 트렌드와 해석이 중요한 작업*이다."

### 실습 3 — 급여대장 분석 Agent (Anthropic API tool use)

payroll\_agent.py — 자연어로 질문하면, 클코가 만들어둔 Python 함수( take\_home\_pay , summarize 등)를 tool로 호출해서 답하는 미니 에이전트.

```

👤 > 김상희의 실수령액은?
🔧 get_employee_details(name='김상희')
🤖 김상희님의 실수령액은 90입니다. 야근비율은 8.3%입니다.

👤 > 야근수당 받는 사람이 몇 명이야?
🔧 get_payroll_summary()
🤖 9명 중 6명 (66.7%)이 야근수당을 받습니다.
  
```

여기서 래리가 짚어준 에이전트의 4요소:

목적 + 가이드 + 뇌(LLM) + 도구(Tool / Skill)

이걸 셋팅하는 방법:

1. 슬래시 /agents 로 만들기 (클코가 에이전트 만드는 에이전트를 띄움)
2. 본인 손으로 직접 markdown 파일로 만들기
3. 추천 — 지금까지 쌓인 세션의 컨텍스트를 그대로 활용해서 "이거 스킬/에이전트로 만들어줘" 하는 게 가장 잘 맞음

## 세션 4 — 웹 스크래핑 (네이버 데이터랩)

`scraper/` 폴더에서 Playwright MCP 플러그인으로 데이터랩을 스크래핑.

산출물 = `data/naver_datahub_scrape_2026-05-16.json` — 쇼핑 인사이트 일별 TOP 10 키워드 + 검색어 트렌드 폼 옵션 메타데이터.

→ 마케터·콘텐츠 PM이 매주 손으로 받던 데이터를 클코가 직접 받아오게 만드는 워크플로의 출발점.

---

## 4. 세션 내내 반복된 메타 룰

---

### a) 라포(rapport) 만들기 — 클로드는 동료다, 그러나 처음 보는 동료다

"이 동료가 DB를 지을 놈처럼 생긴 친구인지 아닌지 모르는 상태에서 막 시키면 안 된다.  
헛소리하는지 동상이몽인지 질문 / 답변 / 요약 / 다시 질문 으로 라포를 잡고 들어가야 한다."

### b) 리뷰 안 하면 stochastic만 쌓인다

매 단계 클로드의 결과물을 내가 수급할 수 있는 형태로 가져오기 (질문, 답변, 요약 중 무엇이든).

"리뷰 안 하면 컨텍스트가 어그러지고, 어그러진 채로 다음 단계로 가면 나중엔 통제 불능."

### c) "내가 결정할 수 있는 건 결정해라"

LLM은 본질적으로 stochastic. 내가 결정할 수 있는 layer(파이썬 버전, 라이브러리, 모델, 컨벤션, 폴더 위치 등)는 다 명시적으로 잡고 들어가야 다음 단계의 noise가 줄어든다.

### d) 회고를 클로드와 함께 — Lessons Learned

세션 끝날 때 마다:

"너랑 나랑 지금까지 한 게 있는데, 레슨런이 뭐야? 좋았던 것 / 아쉬웠던 것 한 줄로 정리해줘."

→ 이 결과를 `CLAUDE.md` 에 누적하면, 다음 세션은 나의 일하는 방식을 알고 시작한다.

### e) `CLAUDE.md` 는 짧게 (400라인 이하)

매 세션 시작에 클로드가 통째로 읽는 파일. 길어지면 토큰 낭비 + 정확도 저하.

### f) 모델 선택은 매번 의식적으로

`/model` 로 확인. 어느 순간 자동으로 다운그레이드되어 있을 수 있다.

원밀리언 컨텍스트는 needle in a haystack 정확도가 약간 떨어지므로, 자동 스위칭에 맡기되 가끔 확인.

## g) 모르면 클로드한테 물어보고, 더 모르면 @bcherny 트위터 보기

"사용법이 아니라 '실제 사람들이 이걸로 뭘 하고 있는지'를 짧게 보여주는 곳이 X(트위터). 강의는 그걸 못 따라간다."

## 5. 우리가 실제로 만든 것 (현재 폴더 ~/larry/ )

```
larry/
├── excel-reader/
│   ├── add.py                # TDD로 만든 두 수 합 함수
│   ├── test_add.py          # parametrized 8 cases
│   ├── test_boundary.py     # IEEE 754, 64bit, NaN/Inf 경계
│   ├── boundary_demo.py     # 경계값 시각화 데모
│   ├── payroll_reader.py    # openpyxl로 급여대장 1단/2단 모두 로드
│   ├── payroll_stats.py     # Employee 집계 (실수령, 야근비율 ...)
│   ├── payroll_export.py    # UTF-8 BOM CSV 출력
│   ├── payroll_agent.py     # Anthropic tool-use 자연어 에이전트
│   ├── 급여대장.xlsx        # 입력
│   ├── payroll_report.csv   # 산출물
│   └── pyproject.toml       # uv + Python 3.14 + openpyxl + anthropic
├── scrapper/
│   └── data/
│       └── naver_datalab_scrape_2026-05-16.json # Playwright MCP 산출물
```

git log:

```
0cd16d2 Add payroll Excel analysis pipeline: reader, stats, CSV export
18425fd Initial commit: add() function with type validation and TDD test suite
```

## 6. 오늘 PM이 가져가는 어휘 — 개발자랑 같은 말 쓰기

| 단어                            | 클코에서 의미                               |
|-------------------------------|---------------------------------------|
| IDE / Editor                  | VS Code, Zed, Cursor 같은 통합작업 환경       |
| Runtime                       | 코드를 실제로 돌려주는 엔진 (Python 3.14 등)       |
| Terminal                      | 사람↔기계 인터페이스. Ghostty/iTerm 같은 게 그 구현체 |
| Shell                         | 터미널 안에서 명령을 해석하는 프로그램 (zsh, bash ...) |
| pwd / ls / cd / mkdir         | 어디에/뭐가/어디로/만들기 — 모든 작업의 4기둥           |
| Repository / Commit / Push    | 작업 단위 저장 → 공유                         |
| Branch / Reset                | 사이드 작업 / 안전한 되돌리기                     |
| Pre-commit hook               | 커밋 직전에 자동으로 도는 검사 (보통 테스트)            |
| Convention / Idiomatic        | 그 도메인에서 "좋은" 방식이라 통하는 것               |
| TDD                           | 테스트 먼저, 통과시키는 코드는 나중                  |
| Boundary test                 | QA 진행할 때, 경계만 잡으면 나머지는 대수적으로 통과       |
| Stochastic / Deterministic    | 확률적 / 결정적 — LLM 쓸지 Python 쓸지의 결정 기준   |
| Intent                        | 프롬프트의 진짜 의도 — 명확할수록 결과가 안정            |
| Plugin / Skill / Agent / Tool | 카탈로그 / 매뉴얼 / 대리인 / 도구                 |
| Harness                       | 목적에 맞게 사람이 채운 '안장' — 셋업 패키지           |
| YOLO mode (permission skip)   | 모든 권한 다 주는 모드 — 위험!                   |

## 7. 참여자의 Next Action Plan (수업 끝나면서 각자 정한 것)

- Joy: 고객사 / 프로젝트별 맥락 쌓기 최적화 (CLAUDE.md, skills 정리)
- Joy: plan mode / goal mode 직접 써보기
- CLAUDE.md 최적화 — 400라인 룰
- 가격비교 크롤링 task — Playwright MCP로 직접 시도
- 매주 손으로 받던 엑셀들 → JSON + 분석 프로그램으로 자동화
- git push / pull 직접 해보기 — clone만 하지 말고 PR도



## 8. Larry에게

---

강의를 진행해주신 김상희 아버지(Larry)께 깊이 감사드립니다.

하루 8시간을 통째로 내주신 것도 감사하지만, 그보다 *"이걸 알았으면 좋았겠다"*의 'why'를 끝까지 짚어가며 가르쳐주신 게 가장 값졌습니다. 각기 다른 이해도, 목표로 앉아있는 구성원들에게 공통으로 / 각자가 필요한 질문들을 다 응답해주셔서 감동했습니다.

같은 자리에 있던 9명의 PM·마케터·사업가가 각자 다른 결과를 만들면서도, 같은 어휘로 일할 수 있게 된 건 전적으로 래리의 *"어떻게 작동하는지 개념에 대한 기본적인 이해, LLM 의 non-deterministic 이라는 성격을 이해한"* 페이스 덕분이라 생각합니다.

도서관 이야기도, 우레탄 공사 이야기도, *"보리스(@bcherny) 트위터 보면 답 나와요"*도 — 다음에도 또 듣고 싶습니다.

감사합니다.

---

— 2026-05-17, 이지이 ([jiyi.lee7@gmail.com](mailto:jiyi.lee7@gmail.com))